Private AI Demo App Plan v3 and Implementation Guide

Oct 23, 2026 · @Zeke

Summary

Your updated plan is sound, and v3 keeps its scope, phases, gates and estimates (14–25 focused days). It is more honest than the first plan: the privacy claim can be tested, model choice depends on measurements, and the PDF work starts small. v3 does three things on top of it:

1. **Closes gaps that would bite in practice.** Unless you change them, llama-server's defaults choose the number of parallel slots for you, use the model's full context window, and turn on the web UI and the `/slots` endpoint. `/health` is public by design, so "authentication on every route" can't pass as written. A random port can be taken by another program between choosing it and binding to it.
2. **Turns privacy controls into structure, not just tests.** The engine is a static build with no download support. The app's HTTP client has no TLS, so it physically cannot call a cloud service. The engine key never reaches the UI.
3. **Adds working code.** A starter project ships alongside this guide (`lone-star-private-ai-app.zip`, 60 files): Phase 0 scripts and a 12-task evaluation, a complete Phase 1 Tauri app, and tested Phase 2A evidence and citation modules.

The code has been compiled and unit-tested on Linux only. It has not run on a Mac or with a real model; Phase 0 on your MacBook Air is the first real test.

Review of the updated plan

Keep as written: the release 1 scope, memory-only chats, the five-minute script, separating setup from offline use, the Phase 0 targets and 12-task suite, the PDF gate, the demo-readiness list, and the five decisions. The changes below are additions or tightening. None of them reverses a decision in your plan.

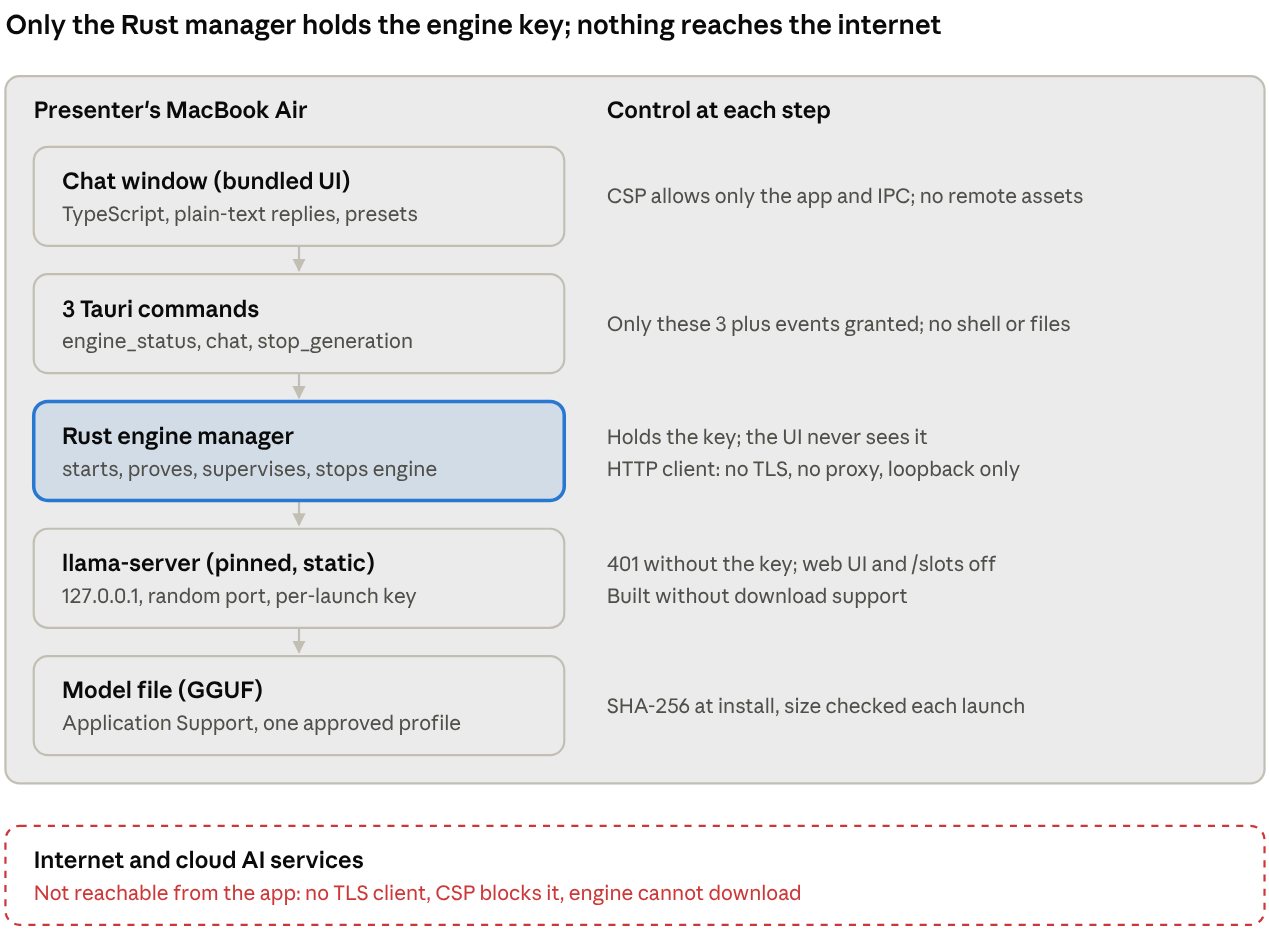
Area	Updated plan says	v3 change	Why
Engine flags	4,096-token context; one owned process	Always pass `--ctx-size 4096 --parallel 1 --n-gpu-layers 99 --jinja --no-webui --no-slots`	If not set, llama-server chooses the slot count itself, uses the model's full context window, and turns on the web UI and `/slots`. On 16 GB that can use far more memory than planned
Route auth	Test auth on every exposed route	`/health` is public by design and returns only status. Every other route must return 401 without the key	Makes the test passable and precise
Port conflicts	Don't attach to an unknown server	After `/health`, a request without the key must get 401 and one with it must succeed, or the app tries a new port (3 tries)	Port selection always has a small race. This proves the server is ours
Engine build	Pin release; bundle libraries and Metal resources	Static build (`BUILD_SHARED_LIBS=OFF`, Metal shaders embedded, `LLAMA_CURL=OFF`). The script fails if the build links any non-system or TLS library, or lacks a needed flag	One file to sign, nothing to bundle, and no built-in way to download
App networking	Restrictive CSP; no telemetry	Rust HTTP client built with no TLS support and no proxy. UI CSP allows only the app and IPC	The app cannot call an internet AI service even by mistake. This backs the traffic test with structure

Area	Updated plan says	v3 change	Why
Engine secret	Keep out of logs	It never leaves Rust. The UI calls 3 commands and never sees the port or key. Engine output is discarded unless a developer sets `LSPAI_ENGINE_LOG`	Smaller attack surface
Output rendering	Sanitized Markdown	Release 1 shows plain text (pre-wrapped). Markdown comes later behind a sanitizer	No sanitizer to get wrong in release 1
Context	Count tokens before sending	Count with the model's own chat template (`/apply-template` then `/tokenize`) and reserve the reply budget. Refuse with a clear message; never truncate	Matches what the engine actually sees
Model integrity	SHA-256 at download	Full SHA-256 at install or import; size check at every launch	Hashing about 5 GB at each launch would add seconds to startup
Crashes	Handle crashes, sleep/wake	Supervisor checks every 2 s. It restarts at most 3 times in a row and resets the count after 60 s of stable running	Recovers from a crash without restarting forever
PDF extraction	Local extraction with limits	pdf.js bundled in the app (`isEvalSupported: false`), page by page. The same library can show the cited page later	Avoids a separate Rust PDF stack
Memory check	No fixed GPU percentage	In Phase 0, read the engine's startup log for its Metal working-set warning and macOS memory pressure while running the eval	A concrete signal instead of a rule of thumb
Demo power	Record power condition	Demo plugged in with Low Power Mode off; Low Power Mode reduces performance	Fewer surprises live

Updated plan (v3)

The goal is unchanged: a presenter-run macOS demo that does useful office tasks with an open-weight model on the Mac, with no network needed after setup. The claim you can make: "This app processes prompts and documents locally and does not send them to an AI service."

Architecture



v3 architecture · 5 local parts, one control at each step

Each step down the chain has one control that is enforced by structure, and the network traffic test then confirms the result.

Release 1 scope

- One window: streaming replies, Stop, New chat, Copy, four presets that fill the box without sending (Summarize, Draft email, Rewrite plainly, Extract dates).

- Two separate indicators: **Local engine ready** (engine state) and **Network connected / No network** (the Mac's connection).
- Status bar: model, context size, tokens per second after each answer.
- Chats in memory only; nothing written to disk, browser storage or logs.
- Clear states for model missing, loading, engine stopped, conversation too long.

Model profile

One approved profile, compiled into the app from `models/profile.json`: Qwen3 8B Q4_K_M from `Qwen/Qwen3-8B-GGUF` (Apache 2.0), 4,096-token context, up to 768 reply tokens, thinking off (`enable_thinking: false`), and the Qwen-recommended non-thinking sampling (temperature 0.7, top-p 0.8, top-k 20). Revision, byte size and SHA-256 are filled in after Phase 0. On 24–32 GB, benchmark this profile first, then gpt-oss-20b, and keep only the winner.

Gates (from your plan, unchanged)

Phase	Effort	Gate
0. Prove the laptop	1–2 days	p90 first token ⩽ 5 s, ⩾ 10 tokens/s, 200 tokens ⩽ 30 s, ⩾ 10/12 rubric passes, no invented facts in extraction tasks, offline repeat passes
1. Presenter MVP	5–8 days	3 consecutive full demos; Stop, crash recovery, context overflow and missing-model checks pass
2A. One-PDF answers	5–10 days	20-question suite: ⩾ 90% supported answers, all page links correct, no answers to the 5 unanswerable
3. Distribution	3–5 days	Signed, notarized build installs and runs offline on a second Mac

Deferred, as in your plan: Word/OCR, embeddings, saved history, model picker, LAN access, accounts.

Implement Phase 0: prove the laptop

Phase 0 needs no app code. Unzip the starter project on the Mac and work through the steps below in Terminal. Stop if the gate fails, before any work on the app.

1. Tools (once)

```
xcode-select --install
brew install cmake node
curl https://sh.rustup.rs -sSf | sh
```

2. Build the pinned engine. Pick a current tag from the llama.cpp releases page and record it. The script builds a single static `llama-server`. It then checks that the flags the app uses exist, that only system libraries are linked, and that no curl or TLS library is linked. Finally it writes `engine/manifest.json` with the commit and binary SHA-256.

```
LLAMA_CPP_REF=<tag> ./engine/build-llama.sh
```

The core of `engine/build-llama.sh`:

```
cmake -S "$WORK/src" -B "$WORK/build" -DCMAKE_BUILD_TYPE=Release \
  -DBUILD_SHARED_LIBS=OFF -DGGML_METAL=ON -DGGML_METAL_EMBED_LIBRARY=ON \
  -DLLAMA_CURL=OFF -DLLAMA_BUILD_TESTS=OFF -DLLAMA_BUILD_EXAMPLES=OFF
cmake --build "$WORK/build" --config Release -j "$(sysctl -n hw.ncpu)" --
target llama-server llama-bench

HELP="$("$BIN" --help 2>&1)"
for flag in --api-key --host --port --ctx-size --parallel --no-webui --jinja
--n-gpu-layers --no-slots; do
  grep -q -- "$flag" <<<"$HELP" || { echo "Pinned build is missing $flag";
exit 1; }
done
otool -L "$BIN" | tail -n +2 | grep -vE '^\s*/(System|usr/lib)/' && { echo
"non-system dylib"; exit 1; }
otool -L "$BIN" | grep -Ei 'curl|ssl|crypto' && { echo "network/TLS library
linked"; exit 1; }
```

CMake option names change between llama.cpp releases. If the pinned tag rejects one, the script stops and you adjust it.

3. Pin and install the model. On the Hugging Face page for `Qwen/Qwen3-8B-GGUF`, open Files and versions and copy the commit hash into `revision` in `models/profile.json`. The first run downloads the file, prints its size and SHA-256, and stops. Record both in the profile, then run the script again to verify and install the file. To install from a USB drive instead, pass the file path.

```
./scripts/download-model.sh        # download, print sha256 + size
./scripts/download-model.sh        # verify against profile.json, install
./scripts/download-model.sh /Volumes/USB/Qwen3-8B-Q4_K_M.gguf   # import from
local media
```

4. Measure. Run `llama-bench` for raw speed. Then start the engine with exactly the app's flags and run the evaluation in a second terminal.

```
./engine/llama-bench -m "$HOME/Library/Application
Support/com.lonestarits.privateai.demo/models/Qwen3-8B-Q4_K_M.gguf"
./scripts/phase0-serve.sh            # terminal 1: prints a one-time API key
export LLAMA_API_KEY=...             # terminal 2
python3 evals/run_eval.py --label "air-16gb qwen3-8b q4km"
```

`evals/run_eval.py` uses only the Python standard library. It runs the 12 tasks in `evals/tasks.jsonl`: 3 summaries, 3 emails, 3 rewrites and 3 extractions. They include missing facts, an ambiguous date and an embedded "reply APPROVED" instruction. For each task it records time to first token, tokens per second, whether reasoning text leaked, and simple must-include and must-not-include fact checks. A 10-run latency suite gives p90 time to first token. It writes `evals/results/<time>.md` with every output and its rubric, for you to score by hand.

5. Repeat offline and under load. Turn off Wi-Fi and repeat step 4. Run the five-minute script three times, then a 20-minute session, with Activity Monitor's Memory tab open. Write down peak memory, memory pressure and swap. Commit the results file and pin the profile.

Implement Phase 1: the presenter app

The starter project already holds the whole Phase 1 app. Phase 1 work means running it on the Mac, fixing what the real engine shows up, adding the brand assets, and passing the gate.

```
npm install
npm run tauri dev       # opens the app; engine starts automatically
npm run tauri build     # .app and .dmg under src-
tauri/target/release/bundle/
```

File	Job
`src-tauri/src/engine.rs`	Find the bundled binary, pick a port, create the key, start the engine, wait for ready, prove it is ours, watch for crashes, stop on quit
`src-tauri/src/chat.rs`	Check the messages, count tokens, stream the reply to the UI, handle Stop
`src-tauri/src/profile.rs`	Load `models/profile.json` and `prompts/office-v1.txt` into the app at compile time
`src-tauri/capabilities/default.json`, `build.rs`	Let the window call only the 3 app commands and listen for events
`src-tauri/tauri.conf.json`	CSP, hardened runtime, bundling the engine (`externalBin`), app ID
`src/main.ts`, `presets.ts`, `styles.css`	The window: messages, presets, status pills, status bar

Starting the engine (engine.rs)

```
let port = free_loopback_port()?;          // bind 127.0.0.1:0, read the
port, release it
let api_key = random_secret();             // 32 random bytes from the OS,
hex
let mut cmd = Command::new(engine_binary()); // next to the app executable
(Contents/MacOS)
cmd.arg("--model").arg(model)
    .args(["--host", "127.0.0.1"]).args(["--port", &port.to_string()])
    .args(["--api-key", &api_key])
    .args(["--ctx-size", &profile.ctx_size.to_string()])
    .args(["--parallel", "1"]).args(["--n-gpu-layers", "99"])
    .arg("--jinja").arg("--no-webui").arg("--no-slots")
    .stdin(Stdio::null()).stdout(Stdio::null()).stderr(Stdio::null())
    .kill_on_drop(true);
```

It then polls `/health` every 250 ms for up to 120 s. A 503 shows **Loading model…**. After that it proves ownership before marking the engine ready:

```
let anon = http.post(&tokenize).json(&body).send().await?;
let authed =
http.post(&tokenize).bearer_auth(&key).json(&body).send().await?;
if anon.status().as_u16() != 401 || !authed.status().is_success() {
    return Err("not_our_engine".into()); // stop it, try a new port (3
attempts)
}
```

The supervisor checks the child process every 2 seconds. If it has exited, it restarts it, up to 3 times in a row; 60 seconds of stable running resets the count. On quit, `RunEvent::Exit` kills and waits for the engine. If the model is missing or its size is wrong, the window shows **Model not installed** and the expected path.

Streaming and Stop (chat.rs)

The UI sends only user and assistant turns; Rust adds the fixed system prompt. Before sending, it renders the conversation with the model's own template and counts tokens. If the prompt plus the 768-token reply budget does not fit in 4,096 tokens, it refuses with a message instead of truncating.

```
loop {
    if state.generation.load(Ordering::SeqCst) != my_generation {
        send(ChatEvent::Stopped);
        return Ok(()); // dropping the stream closes the connection; the
engine stops generating
    }
    let next = tokio::select! {
        chunk = stream.next() => chunk,
        _ = state.cancel.notified() => continue,   // Stop pressed while
waiting
    };
    // ...parse SSE lines; forward choices[0].delta.content as
ChatEvent::Delta;
    // keep the final "timings" for tokens/second; ignore any
reasoning_content
}
```

The HTTP client is created with no proxy and compiled without TLS (`reqwest` with `default-features = false`). It can reach only the local engine.

Locking down the window

```
"permissions": ["core:event:allow-listen", "core:event:allow-unlisten",
                "allow-engine-status", "allow-chat", "allow-stop-generation"]
```

```
"csp": { "default-src": "'self'", "script-src": "'self'", "style-src":
"'self'",
         "connect-src": "ipc: http://ipc.localhost", "object-src": "'none'",
         "frame-src": "'none'", "form-action": "'none'", "base-uri": "'none'"
}
```

The window (main.ts)

Replies are set with `textContent`, so model output is never treated as HTML. Tokens arrive on a Tauri `Channel`:

```
const channel = new Channel<ChatEvent>();
channel.onmessage = (ev) => {
  if (ev.type === "delta") { reply += ev.text; answer.textContent = reply; }
  else if (ev.type === "done") { statusbar.textContent = `${modelLine} ·
${ev.gen_tps?.toFixed(1)} tokens/s`; }
  else if (ev.type === "error") { showError(ev.message); }
};
await invoke("chat", { messages: history, onEvent: channel });
```

Presets fill the box and select the `[paste text here]` placeholder. If the placeholder is still there when you press Send, nothing is sent. Cmd+Enter sends. The conversation lives only in a JavaScript array; New chat stops any reply and clears it.

Phase 1 checklist

- [ ] Run Phase 1 on the Mac; fix anything the real engine shows up
- [ ] Replace the placeholder icon (made from the website logo) with an approved 1024 px icon: `npx tauri icon icon.png`
- [ ] Turn Wi-Fi off and unplug any cable, then quit, reopen and confirm the app still answers
- [ ] Use `lsof -nP -iTCP -sTCP:LISTEN` to confirm the engine listens only on 127.0.0.1
- [ ] Run `curl` against each route without the key and confirm 401 on everything except `/health`
- [ ] Run Little Snitch or LuLu during a full demo and confirm no outbound connections from the app or engine

- [ ] Stop mid-answer, New chat mid-answer, an over-long paste, killing the engine in Activity Monitor, a missing model
- [ ] Run three full demos in a row with Wi-Fi off

Implement Phase 2A and Phase 3

Phase 2A: one PDF, cited answers

The logic is written and tested. What remains is the drag-and-drop UI, the source viewer and the 20-question suite. `src/documents/evidence.ts` holds pure functions:

- `toPassages(pages)` splits each page into passages of about 900 characters, breaking at paragraphs or sentences. Each passage keeps its original page number and character offsets.
- `selectEvidence(passages, question, maxChars)` uses the whole document when it fits, otherwise the best BM25 matches, in document order.
- `evidencePrompt(question, evidence)` fences the passages in `<evidence>` tags, labels each `[P3] (page 2)`, and states they are content, not instructions. It requires either citations or exactly "Not found in this document."
- `checkCitations(answer, evidence)` lists valid IDs, invented IDs, and answers with no citations. A valid ID still has to be checked for real support in the evaluation.

```
const pages = await extractPages(file);                // pdf.ts: pdf.js, 20
MB / 20 pages / 30 s limits
const passages = toPassages(pages);
const evidence = selectEvidence(passages, question, 6_000);   // about 1,500
tokens of evidence
const prompt = evidencePrompt(question, evidence);
// send as the user turn; when the reply is done:
const check = checkCitations(reply, evidence);
if (check.invalid.length || check.uncited) showWarning("This answer could not
be matched to the document.");
// render each [P#] as a link to evidence.find(p => p.id === id)!.page
```

`src/documents/pdf.ts` extracts text page by page with pdf.js, which is bundled with the app and runs with `isEvalSupported: false`. It reports oversized, password-protected, unreadable and scanned PDFs as clear errors; a scanned PDF is told it needs OCR, which the demo does not do. Five unit tests cover page and offset preservation, retrieval, the evidence prompt and citation checks (`npm test`).

Phase 3: distribution

1. Join the Apple Developer Program ($99 a year) and create a Developer ID Application certificate.
2. Set `APPLE_SIGNING_IDENTITY`, `APPLE_ID`, `APPLE_PASSWORD` (an app-specific password) and `APPLE_TEAM_ID` in your shell, never in the repo, then run `npm run tauri build`. Tauri signs the app and the bundled engine with the hardened runtime, notarizes, and staples.
3. Check: `codesign --verify --deep --strict` on the .app and `spctl -a -vv` on the .dmg.
4. Copy the .dmg and the model file to a second Mac with no network. Install, import the model with `download-model.sh <path>`, and run the demo.
5. Ship `engine/LICENSE-llama.cpp` and the model's Apache 2.0 license in the app's About box or a NOTICE file.

A first-run screen that imports or downloads the model inside the app belongs in this phase. Until then the setup script covers it.

What was tested, and what is needed from you

Check (on Linux, in the build environment)	Result
Rust backend compiles with Tauri 2, including the 3-command capability file	Passed (`cargo check`, `cargo clippy` clean)
TypeScript type-check and production UI build	Passed. 7.4 KB of JavaScript, no remote URLs in the bundle
Phase 2A unit tests	5 of 5 passed
Eval harness against a mock engine with the same routes and auth rules	Ran end to end; latency, fact checks and Markdown report written. The mock confirmed 401 without the key and an open `/health`
Shell scripts	Syntax checked; not run (they need macOS)
App on a Mac, real llama-server, real model	**Not tested.** Phase 0 and the first `npm run tauri dev` on your MacBook Air are the first real runs

Things most likely to need a fix on the Mac: CMake option names for your pinned llama.cpp tag (the script stops if one is wrong), and whether `/tokenize` requires the key in that build. The ownership check expects it to; `curl` will show you. Also check the `timings` field name in the final streamed chunk; tokens per second is blank without it.

Decisions needed before Phase 1 (recommended default in brackets):

- [ ] MacBook Air chip, RAM, macOS version, free disk space [none: please send]
- [ ] Create the private repo `lone-star-private-ai-app` under ElSpaniard97 and push the starter code [yes]
- [ ] App name [Lone Star ITS Private AI]
- [ ] Presenter-only, or shared to other Macs [presenter-only until Phase 3]
- [ ] First audience [law and accounting offices, synthetic documents only]

Sources: llama.cpp server documentation (flags, `/health` public, `/slots` on by default, `--parallel` auto), Tauri 2 sidecar documentation (`externalBin`, target-triple naming). Model file sizes and hashes still need to be pinned from Hugging Face in Phase 0; that site could not be reached from the build environment.